Juan González Márquez - C4F588, Jeferson Marín Porras - C24549,
Ariadna Ardila Gutiérrez - C4C646, Juan Sebastán Loazia Ospina B74200

File System - Versión actualizada

El diseño propuesto en el presente documento corresponde a un file system pensado para operar junto con el servidor del proyecto. Este permite almacenar y gestionar información de figuras LEGO y sus componentes.

El sistema de archivos se implementa utilizando un archivo en formato JSON, el cual representa de manera estructurada todos los componentes necesarios para simular un sistema de almacenamiento. Sobre este archivo se define un modelo basado en bloques que permite organizar, almacenar y recuperar la información de forma controlada.

I. Organización general

El file system se divide en bloques de tamaño fijo de 256 bytes, los cuales representan la unidad mínima de almacenamiento. Dichos bloques no corresponden a divisiones físicas del archivo JSON, sino a una estructura manejada por el programa que manipula el sistema de archivos.

La estructura general del JSON se compone de:

- Un “superbloque” con información global
- Un conjunto de bloques identificados de forma única
- Un directorio principal
- Estructuras tipo “figure_data” para representar figuras
- Segmentos y piezas como unidades de contenido

A continuación, se muestra un ejemplo simplificado de cómo luce el file system:

```json
{
  "superblock": {
    "block_size": 256,
    "total_blocks": 1000,
    "free_blocks": [40, 41, 42],
    "root_directory": 0
  },

  "blocks": {
    "0": {
      "type": "directory",
      "name": "/",
      "figures": [
        { "name": "carro", "figure_data": 10 }
      ],
      "logs": [30, 31, 32]
    },

    "10": {
      "type": "figure_data",
      "name": "carro",
      "segments": [11],
      "metadata": {
        "total_pieces": 30
      }
    },

    "11": {
      "type": "segment",
      "pieces": [20]
    },

    "20": {
      "type": "piece",
      "name": "brick 2x2",
      "quantity": 10
    },

    "30": {
        "type": "log",
        "from": "cliente_1",
        "to": "intermediario_1",
        "message_type": "LIST_FIGURES"
    },

    "31": {
        "type": "log",
        "from": "intermediario_2",
        "to": "servidor_2",
        "message_type": "ASK_FIGURE",
        "figure": "carro"
    }
  }
}
```

II. Superbloque

El superbloque contiene la información global del sistema:

- Tamaño de bloque (256 bytes)
- Cantidad total de bloques
- Lista de bloques libres
- Referencia al directorio principal

Este bloque permite administrar el estado del sistema y gestionar la asignación de espacio.

```
{
  "superblock": {
    "block_size": 256,
    "total_blocks": 1000,
    "free_blocks": [30, 31, 32],
    "root_directory": 0
  },
```

III. Directorio principal

El directorio principal se implementa como un bloque de tipo “directory”, ubicado en una posición conocida (por ejemplo, el bloque 0). Este contiene referencias a las figuras almacenadas, relacionando nombres con sus respectivos inodos.

```
"blocks": {
  "0": {
    "type": "directory",
    "name": "/",
    "figures": [
      { "name": "carro", "inode": 10 }
    ]
  },
```

El directorio puede crecer al agregar nuevas entradas, por lo que éste también puede distribuirse en múltiples bloques de ser necesario.

IV. Bloques de figuras

Cada figura se representa mediante un bloque tipo “figure_data”, el cual contiene:

- Nombre de la figura
- Lista de elementos asociados
- Metadatos (como cantidad total de piezas)

```
"10": {
  "type": "inode",
  "name": "carro",
  "segments": [11],
  "metadata": {
    "total_pieces": 30
  }
},
```

Esto permite separar la información estructural del contenido real de la figura.

V. Segmentos y piezas

Los segmentos agrupan piezas, permitiendo distribuir la información en múltiples bloques. Los segmentos y piezas se representan de la siguiente manera:

```
"11": {
  "type": "segment",
  "pieces": [20]
},

"20": {
  "type": "piece",
  "name": "brick 2x2",
  "quantity": 10
}
```

VI. Crecimiento de las estructuras

El sistema permite el crecimiento dinámico de las estructuras mediante la asignación de nuevos bloques.

- Si un inode requiere más segmentos, se agregan nuevos bloques.
- Si un segmento contiene demasiadas piezas, puede dividirse en múltiples bloques.
- Las piezas pueden distribuirse en varios segmentos si es necesario.

Cuando una estructura excede la capacidad de un bloque, su contenido se divide y se enlaza mediante referencias a otros bloques.

VII. Almacenamiento de una imagen

Las imágenes se almacenan como secuencias de datos divididas en múltiples bloques.

1. La imagen se convierte en una secuencia de bytes.
2. Se divide en fragmentos de tamaño limitado.
3. Cada fragmento se almacena en un bloque tipo “data”.
4. Un bloque índice mantiene la referencia a todos los segmentos de la imagen.

VIII. Manejo de bloques libres

El sistema mantiene una lista de bloques disponibles dentro del superbloque que luce de la siguiente manera:

```
"free_blocks": [30, 31, 32],
```

Cuando se necesita almacenamiento:

- Se toma un bloque libre
- Se asigna a la estructura correspondiente
- Se actualiza la lista de bloques libres

IX. Control del tamaño de bloque

El tamaño de 256 bytes por bloque se define como una restricción lógica, controlada por el programa que manipula el sistema de archivos.

1. Escritura de datos

Cuando el programa almacena información:

- Divide los datos en fragmentos cuyo tamaño no exceda el límite permitido.
- Cada fragmento se guarda en un bloque independiente.
- Si la información es mayor, se distribuye en múltiples bloques enlazados.

2. Lectura de datos

Durante la lectura:

- El programa carga completamente el archivo JSON en memoria.
- Accede a los bloques como estructuras lógicas.
- Recupera el contenido completo de cada bloque.
- Reconstruye la información original siguiendo las referencias entre bloques.

El sistema no lee el archivo JSON en segmentos físicos de 256 bytes, sino que trabaja con bloques lógicos previamente estructurados. Debido a que el control de tamaño se realiza en la escritura, cada bloque leído ya cumple con la restricción establecida.

X. Sistema de logs

El sistema también incluye un registro de operaciones para facilitar el seguimiento de acciones y el análisis del programa.

```
"30": {
    "type": "log",
    "from": "cliente_1",
    "to": "intermediario_1",
    "message_type": "LIST_FIGURES"
},

"31": {
    "type": "log",
    "from": "intermediario_2",
    "to": "servidor_2",
    "message_type": "ASK_FIGURE",
    "figure": "carro"
},
```

Esta sección también puede dividirse en varios bloques si el tamaño del registro crece. Asimismo, el directorio principal mantiene una referencia a esta sección, indicando los bloques donde se encuentra almacenada la información de los logs.